the rest of your application. This leads to more maintainable, testable, and reusable code.

For example, imagine you have a controller that needs to fetch active users. Without the repository pattern, you might write a query directly in the controller:

```
$users = DB::table('users')->where('active', 1)->get();
```

With a repository, this logic is encapsulated in a method:

```
$users = $userRepository->getActiveUsers();
```

This separation means you can easily swap out the underlying implementation (e.g., switch from Query Builder to Eloquent or an API) without changing the controller. It also improves unit testing, as you can mock repository interfaces instead of depending on the actual database.

11.6 Exercise 32: Performing CRUD with Query Builder in Laravel 12

11.6.1 Description

In this lab, we will build a simple **User Management Web Application** using Laravel 12. Instead of using Eloquent, we will use **Laravel's Query Builder** to perform CRUD (Create, Read, Update, Delete) operations. The UI will be styled using **Bootstrap 5** for a better visual experience.

11.6.2 Objectives

- Understand how to use Laravel Query Builder to interact with the database.
- Build a web-based CRUD interface using Laravel and Blade templates.
- Apply Bootstrap to create a clean and responsive UI.

11.6.3 Prerequisites

Before you start, make sure you have the following:

- Laravel 12 installed globally
- A local or remote MySQL/MariaDB server or SQLite database
- PHP 8.2 or higher